



Implementação de PKI com HSM

Autor:
Leonardo Oliveira Feitosa

1. Introdução:

Este laboratório tem como objetivo demonstrar, na prática, a implementação dos principais conceitos de Infraestrutura de Chaves Públicas (PKI) e gerenciamento de certificados digitais em um ambiente controlado. Durante as etapas do experimento, foi realizada a criação de uma Autoridade Certificadora (CA) utilizando o OpenSSL, com a chave privada protegida por um Hardware Security Module (HSM) simulado através do SoftHSM utilizando o padrão PKCS#11.

A partir dessa estrutura, foram gerados e assinados certificados digitais para um servidor web, possibilitando a configuração de comunicação segura via HTTPS em um servidor Apache HTTP Server. O laboratório busca, portanto, integrar conceitos de criptografia aplicada, gerenciamento seguro de chaves e certificados, demonstrando como essas tecnologias são utilizadas em ambientes corporativos para garantir autenticidade, confidencialidade e integridade nas comunicações de rede.

2. Arquitetura do Laboratório:



3. Preparação do Ambiente:

```
Shell  
sudo apt update && apt upgrade -y
```

Atualizar o sistema.

```
Shell  
sudo apt install -y \  
softhsm2 \  
opensc \  
libengine-pkcs11-openssl \  
openssl \  
apache2 \  

```

Instalar os serviços e ferramentas para o laboratório.

4. HSM:

4.1 - Inicializar o token HSM (inserir o PIN).

```
Shell
softhsm2-util --init-token \
--free \
--label "PKI-HSM"
```

```
Shell
softhsm2-util --show-slots
```

Verificar o token criado.

4.2 Gerar chave da CA dentro do HSM.

```
Shell
pkcs11-tool \
--module /usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so \
--login \
--pin 1234 \
--keypairgen \
--key-type rsa:2048 \
--id 01 \
--label CAkey
```

```
Shell
pkcs11-tool \
--module /usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so \
--login \
--pin 1234 \
--list-objects
```

Listar a chave.

4.3 Gerar CSR usando a chave do HSM.

```
Shell
export PKCS11_MODULE_PATH=/usr/lib/x86_64-linux-gnu/softhsm/libsofthsm2.so
export PKCS11_PIN=1234

openssl req \
-engine pkcs11 \
-keyform engine \
-new \
```

```
-key "pkcs11:object=CAkey;type=private" \  
-out ca.csr \  
-config ca.cnf
```

4.4 Criar certificado da Root CA.

```
Shell  
openssl x509 \  
-req \  
-in ca.csr \  
-engine pkcs11 \  
-keyform engine \  
-signkey "pkcs11:object=CAkey;type=private" \  
-extfile ca-ext.cnf \  
-days 3650 \  
-out ca.crt
```

```
Shell  
openssl x509 -in ca.crt -text -noout
```

Verificar certificado.

4.5 Criar CSR do servidor.

```
Shell  
openssl req -new \  
-newkey rsa:2048 \  
-nodes \  
-keyout server.key \  
-out server.csr
```

CN = lab-server.

4.6 Assinar certificado do servidor com a CA.

```
Shell  
openssl ca \  

```

```
-config openssl-ca.cnf \  
-engine pkcs11 \  
-keyform engine \  
-in server.csr \  
-out server.crt \  
-days 365
```

```
Shell  
openssl verify -CAfile ca.crt server.crt
```

Verificar certificado.

4.7 Instalar CA no sistema.

```
Shell  
sudo cp ca.crt /usr/local/share/ca-certificates/lab-root-ca.crt  
sudo update-ca-certificates
```

Evidências:

4.1

```
root@debian:/home/debian# softhsm2-util --init-token --slot 0 --label "PKI-HSM"
=== SO PIN (4-255 characters) ===
Please enter SO PIN: ****
Please reenter SO PIN: ****
=== User PIN (4-255 characters) ===
Please enter user PIN: ****
Please reenter user PIN: ****
The token has been initialized and is reassigned to slot 596206149
root@debian:/home/debian#
```

4.2

```
root@debian:/home/debian# pkcs11-tool \
--module /usr/lib/softhsm/libsofthsm2.so \
--login \
--pin 1234 \
--keypairgen \
--key-type rsa:2048 \
--label CAkey \
--id 01
Using slot 0 with a present token (0x23896245)
Key pair generated:
Private Key Object; RSA
  label:      CAkey
  ID:         01
  Usage:      decrypt, sign, unwrap
  Access:     sensitive, always sensitive, never extractable, local
Public Key Object; RSA 2048 bits
  label:      CAkey
  ID:         01
  Usage:      encrypt, verify, wrap
  Access:     local
root@debian:/home/debian#
```


4.3

```
root@debian:/home/debian# openssl engine -t
(dynamic) Dynamic engine loading support
[ unavailable ]
root@debian:/home/debian# openssl req \
-new \
-engine pkcs11 \
-keyform engine \
-key "pkcs11:object=CAkey;type=private" \
-out ca.csr
Engine "pkcs11" set.
Enter PKCS#11 token PIN for PKI-HSM:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:br
State or Province Name (full name) [Some-State]:sp
Locality Name (eg, city) []:sp
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:lab-root-ca
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
```

4.4

```
root@debian:/home/debian# openssl x509 \
-req \
-in ca.csr \
-engine pkcs11 \
-keyform engine \
-key "pkcs11:object=CAkey;type=private" \
-days 3650 \
-out ca.crt
Engine "pkcs11" set.
Enter PKCS#11 token PIN for PKI-HSM:
Certificate request self-signature ok
subject=C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
root@debian:/home/debian#
```

4.5

```
debian@debian: ~  
root@debian:/home/debian# openssl genrsa -out server.key 2048  
root@debian:/home/debian# openssl req \  
-new \  
-key server.key \  
-out server.csr  
You are about to be asked to enter information that will be incorporated  
into your certificate request.  
What you are about to enter is what is called a Distinguished Name or a DN.  
There are quite a few fields but you can leave some blank  
For some fields there will be a default value,  
If you enter '.', the field will be left blank.  
-----  
Country Name (2 letter code) [AU]:br  
State or Province Name (full name) [Some-State]:sp  
Locality Name (eg, city) []:sp  
Organization Name (eg, company) [Internet Widgits Pty Ltd]:  
Organizational Unit Name (eg, section) []:  
Common Name (e.g. server FQDN or YOUR name) []:lab-server  
Email Address []:  
  
Please enter the following 'extra' attributes  
to be sent with your certificate request  
A challenge password []:  
An optional company name []:
```

4.6

```
debian@debian: ~  
root@debian:/home/debian# openssl ca \  
-config openssl-ca.cnf \  
-engine pkcs11 \  
-keyform engine \  
-in server.csr \  
-out server.crt \  
-days 365  
Engine "pkcs11" set.  
Using configuration from openssl-ca.cnf  
Enter PKCS#11 token PIN for PKI-HSM:  
Check that the request matches the signature  
Signature ok  
The Subject's Distinguished Name is as follows  
countryName :PRINTABLE:'br'  
stateOrProvinceName :ASN.1 12:'sp'  
localityName :ASN.1 12:'sp'  
organizationName :ASN.1 12:'Internet Widgits Pty Ltd'  
commonName :ASN.1 12:'lab-server'  
Certificate is to be certified until Mar 12 06:54:20 2027 GMT (365 days)  
Sign the certificate? [y/n]:y  
  
1 out of 1 certificate requests certified, commit? [y/n]y  
Write out database with 1 new entries  
Database updated  
root@debian:/home/debian# ls -l server.crt  
-rw-r--r-- 1 root root 3869 Mar 12 01:54 server.crt  
root@debian:/home/debian#
```

```
debian@debian: ~  
root@debian:/home/debian# openssl verify -CAfile ca.crt server.crt  
server.crt: OK  
root@debian:/home/debian# |
```

4.7

```
debian@debian: ~  
root@debian:/home/debian# sudo cp ca.crt /usr/local/share/ca-certificates/lab-ca.crt  
root@debian:/home/debian# sudo update-ca-certificates  
Updating certificates in /etc/ssl/certs...  
rehash: warning: skipping ca-certificates.crt, it does not contain exactly one certificate or CRL  
1 added, 0 removed; done.  
Running hooks in /etc/ca-certificates/update.d...  
done.  
root@debian:/home/debian# ls /etc/ssl/certs | grep lab  
lab-ca.pem  
root@debian:/home/debian# |
```

5. Apache com certificados:

Editar o arquivo de configuração em “/etc/apache2/sites-available/default-ssl.conf” e adicionar as informações abaixo:

```
Shell
SSLEngine on

SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key
SSLCertificateChainFile /etc/apache2/ssl/ca.crt
```

Criar um diretório de certificados e copiar os certificados para o diretório, ativar o SSL e reiniciar o serviço:

```
Shell
sudo mkdir /etc/apache2/ssl
sudo cp server.crt /etc/apache2/ssl/
sudo cp server.key /etc/apache2/ssl/

sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl restart apache2
```

Testar apache com os certificados:

```
Shell
curl -k https://localhost

openssl s_client -connect localhost:443
```

Saída esperada, “Verify return code: 0 (ok)”.

Evidências:

```
debian@debian: ~  
GNU nano 7.2 /etc/apache2/sites-available/default-ssl.conf  
# SSL Engine Switch:  
# Enable/Disable SSL for this virtual host.  
SSLEngine on  
  
# A self-signed (snakeoil) certificate can be created by installing  
# the ssl-cert package. See  
# /usr/share/doc/apache2/README.Debian.gz for more info.  
# If both key and certificate are stored in the same file, only the  
# SSLCertificateFile directive is needed.  
SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem  
SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key  
SSLCertificateFile /etc/apache2/ssl/server.crt  
SSLCertificateKeyFile /etc/apache2/ssl/server.key  
SSLCertificateChainFile /etc/apache2/ssl/ca.crt
```

Arquivo de configuração.

```
debian@debian: ~  
root@debian:/home/debian# curl https://localhost  
curl: (60) SSL: certificate subject name 'lab-server' does not match target host name 'localhost'  
More details here: https://curl.se/docs/sslcerts.html  
  
curl failed to verify the legitimacy of the server and therefore could not  
establish a secure connection to it. To learn more about this situation and  
how to fix it, please visit the web page mentioned above.  
root@debian:/home/debian# curl -k https://localhost  
  
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1  
-transitional.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
  <head>  
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />  
    <title>Apache2 Debian Default Page: It works</title>  
    <style type="text/css" media="screen">
```

Local host.

```
-----END CERTIFICATE-----  
subject=C = br, ST = sp, O = Internet Widgits Pty Ltd, CN = lab-server  
issuer=C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca  
-----  
No client certificate CA names sent  
Peer signing digest: SHA256  
Peer signature type: RSA-PSS  
Server Temp Key: X25519, 253 bits  
-----  
SSL handshake has read 2215 bytes and written 377 bytes  
Verification: OK  
-----  
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384  
Server public key is 2048 bit  
Secure Renegotiation IS NOT supported  
Compression: NONE  
Expansion: NONE  
No ALPN negotiated  
Early data was not sent  
Verify return code: 0 (ok)
```

```

root@debian:/home/debian# openssl s_client -connect localhost:443
CONNECTED(00000003)
Can't use SSL_get_servername
depth=1 C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
verify return:1
depth=0 C = br, ST = sp, O = Internet Widgits Pty Ltd, CN = lab-server
verify return:1
---
Certificate chain
 0 s:C = br, ST = sp, O = Internet Widgits Pty Ltd, CN = lab-server
  i:C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
  a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Mar 12 06:54:20 2026 GMT; NotAfter: Mar 12 06:54:20 2027 GMT
 1 s:C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
  i:C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
  a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Mar 12 06:46:31 2026 GMT; NotAfter: Mar  9 06:46:31 2036 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDJzCCAq8CAhAAMA0GCSqGSIb3DQEBCwUAMGAxCzAJBgNVBAYTAmJyMQswCQYD
VQQIDAjZcDELMAkGA1UEBwwCc3AxITAFBgNVBAoMGEUdGVybWV0IFdpZGdpdHMg
UHR5IEEx0ZDEUMBIGA1UEAwwLbGFiLXJvb3QtY2EwHhcNMjYwMzEyMDY1NDIwWWhcN
MjcwMzEyMDY1NDIwWjBSMQswCQYDVQGEwJicjELMAkGA1UECAwCc3AxITAFBgNV
BAoMGEUdGVybWV0IFdpZGdpdHMgUHR5IEEx0ZDEUMBIGA1UEAwwLbGFiLXNlcnZl
cjcCCASiWDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAJGuFdzrg0YtxSbr51WP
uWqFFqt1A/RMhQJLNq6VuosQ7UGYVhUhzF9WzLacfqAi6/ESakzY9f1NSY3qXgZi
DYUuGwQCdAH03i+DMDfh0jeTzUNcP/PDCguGVjFVX+q6YAKjG0sSCDWxqyTcUwkG
EQuziXzPBIUNkyl1KNxACHTHWwrBD0xjjfbV08BjkMGVv1ZF0ciV0qLZWw1d9xBc
j88JtU5D1HnDAd10c/hWWhnCwDfQ0S7zw6cFFieV3/T7ZLh7Q1r8AgQiRt2vkGms
byTsZMsWtmYvtbKJAiIN+9IqWDPjULjtwbMLKaY0gPoGn4KqFwjdxTgLNuyap
uGcCAwEAATANBgkqhkiG9w0BAQsFAAOCQAQEAfKJlogPiWYMLGAKUYMLF3w89y/J
uZJwY2LZpqy7z15xiXeNamycTsNTf/PLT7LE00JUo6aw0b/4ikvhEDYPXae1Js2d
r1Uymo3rdSjuB7UI48xXBzRmIoaX029e1UKh49/2RDGiqqDXYSOGFumhiHlw00Fg
h5y9qP6T4zx3q1ophrQJkCtFHU9CBjw040iEWB4/kRkSqtQnqUgI2HBhzmOCm3+2
CmzEH2Hkxa+GT6JCG09Vv9KgYarH8ywknR1ZRBsELKs6XG3oEC7KNI8VTfWbGHkD
X/sNHQVAQITa8dmqNVBxpKGJb9x0Xhw+ccle2kkUc7zrp1yDZgwU8HnsPg==
-----END CERTIFICATE-----

```

Validação do certificado no apache.

6. Vault como Intermediate CA:

O Vault está atuando como a autoridade certificadora intermediária, emitindo certificados TLS para aplicações. Ele automatiza a geração, distribuição e revogação de certificados usando a infraestrutura PKI criada, assim os serviços do lab podem obter certificados válidos sem precisar gerar ou assinar manualmente na CA.

```
Shell
vault server -dev
```

Abrir o servidor para pegar as credenciais.

```
Shell
export VAULT_ADDR='http://127.0.0.1:8200'

export VAULT_TOKEN="SEU_ROOT_TOKEN"
```

Logar.

```
Shell
vault secrets enable -path=pki_int pki

vault secrets tune -max-lease-ttl=43800h pki_int
```

Criando a PKI para a Intermediate CA.

```
Shell
vault write -format=json pki_int/intermediate/generate/internal \
common_name="Lab Intermediate CA" \
ttl=43800h > intermediate.json

jq -r '.data.csr' intermediate.json > intermediate.csr
```

Gerando e extraíndo o CSR.

```
Shell
openssl ca \
-config openssl-ca.cnf \
-engine pkcs11 \
-keyform engine \
-in intermediate.csr \
-out intermediate.crt \
-days 1825 \
-extfile intermediate-ext.cnf
```

Assinar a Intermediate com a Root (HSM).

```
Shell
vault write pki_int/intermediate/set-signed \
certificate=@intermediate.crt
```

Importar a Intermediate no Vault.

```
Shell
vault write pki_int/config/urls \
issuing_certificates="http://127.0.0.1:8200/v1/pki_int/ca" \
crl_distribution_points="http://127.0.0.1:8200/v1/pki_int/crl"
```

Configurar URLs da PKI.

```
Shell
vault write pki_int/roles/web \
allowed_domains="lab.local" \
allow_subdomains=true \
max_ttl="72h"
```

Criar role para emissão de certificados.

```
Shell
vault write -format=json pki_int/issue/web \
common_name="app.lab.local" > cert.json
```

Emitir um certificado.

```
Shell
jq -r '.data.certificate' cert.json > app.crt

jq -r '.data.private_key' cert.json > app.key

jq -r '.data.issuing_ca' cert.json > ca_chain.crt
```

Extrair certificados.

```
Shell
openssl x509 -in app.crt -text -noout
```

Verificar o certificado.

```
Shell
SSLEngine on
```




```
SSLCertificateFile /home/debian/app.crt  
SSLCertificateKeyFile /home/debian/app.key  
SSLCertificateChainFile /home/debian/ca_chain.crt
```

Usar os certificados novos no apache e depois reiniciar o serviço.

Revogar um certificado:

```
Shell  
openssl x509 -in app.crt -noout -serial
```

Obter o serial number do certificado.

```
Shell  
vault write pki_int/revoke serial_number="serial_number"
```

Revogando no vault, com isso o certificado deixa de ser confiável.

```
Shell  
curl http://127.0.0.1:8200/v1/pki_int/crl -o crl.pem
```

Baixar a lista de certificados revogados.

Evidências:

```
debian@debian: ~  
2026-03-12T03:53:15.958-0500 [INFO] identity: entities restored  
2026-03-12T03:53:15.958-0500 [INFO] identity: groups restored  
2026-03-12T03:53:15.958-0500 [INFO] expiration: lease restore complete  
2026-03-12T03:53:15.959-0500 [INFO] core: post-unseal setup complete  
2026-03-12T03:53:15.959-0500 [INFO] core: vault is unsealed  
2026-03-12T03:53:15.962-0500 [INFO] core: successful mount: namespace="" path=secret/ type=kv version  
="v0.25.0+builtin"  
WARNING! dev mode is enabled! In this mode, Vault runs entirely in-memory  
and starts unsealed with a single unseal key. The root token is already  
authenticated to the CLI, so you can immediately begin using Vault.  
  
You may need to set the following environment variables:  
  
$ export VAULT_ADDR='http://127.0.0.1:8200'  
  
The unseal key and root token are displayed below in case you want to  
seal/unseal the Vault or re-authenticate.  
  
Unseal Key: N9d/H/lCa+Tb3NMp1pfNVH1yLWJLm/q1mWmFc1pWpR0=  
Root Token: hvs.vxWd0VIWD9TXawHIUb99gxt9  
  
Development mode should NOT be used in production installations!  
  
root@debian:/home/debian#  
root@debian:/home/debian# export VAULT_ADDR='http://127.0.0.1:8200'  
root@debian:/home/debian# export VAULT_TOKEN="hvs.vxWd0VIWD9TXawHIUb99gxt9"  
root@debian:/home/debian# vault status  
Key          Value  
----  
Seal Type    shamir  
Initialized   true  
Sealed        false  
Total Shares  1  
Threshold     1  
Version       1.21.4  
Build Date    2026-03-04T17:40:05Z  
Storage Type  inmem  
Cluster Name  vault-cluster-d2230abe  
Cluster ID    414f7126-deba-57c6-d587-39a298f0a0f0  
HA Enabled    false  
root@debian:/home/debian#
```

Configurando a intermediate.

```

debian@debian: ~
2026-03-12T03:53:15.962-0500 [INFO] core: successful mount: namespace="" path=secret/ type=kv version
="v0.25.0+builtin"
WARNING! dev mode is enabled! In this mode, Vault runs entirely in-memory
and starts unsealed with a single unseal key. The root token is already
authenticated to the CLI, so you can immediately begin using Vault.

You may need to set the following environment variables:

$ export VAULT_ADDR='http://127.0.0.1:8200'

The unseal key and root token are displayed below in case you want to
seal/unseal the Vault or re-authenticate.

Unseal Key: N9d/H/LCa+Tb3NMp1pfNVH1yLWJLm/q1mWmFc1pWpR0=
Root Token: hvs.vxWd0VIWD9TXawHIUb99gxt9

Development mode should NOT be used in production installations!

2026-03-12T03:57:06.470-0500 [INFO] secrets.pki.pki_9e789d5f: 9c9fbec5-2fb1-994d-2634-b79038874b9b: s
ucceeded in migrating to issuer storage version 2
2026-03-12T03:57:06.470-0500 [INFO] core: successful mount: namespace="" path=pki_int/ type=pki versi
on="v1.21.4+builtin.vault"
2026-03-12T03:57:15.815-0500 [INFO] core: mount tuning of leases successful: path=pki_int/

debian@debian: ~
updating /config/urls or the newly generated issuer with this information.
Since this certificate is an intermediate, it might be useful to regenerate
this certificate after fixing this problem for the root mount.

Key      Value
----
csr      -----BEGIN CERTIFICATE REQUEST-----
MIICYZCCAUsCAQAwHjEcmBoGA1UEAxMTTGFiIEludGVybWVkaWF0ZSBBDQTCASIW
DQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAA00MbxaytJwpGniEKkUvXDuv5L6m
bEQ/BefPMwEBLAvhUmBDMm560ema6wrwzvpeKNqE9to94ziJhdF3xu/3RwlgfASo
88X0DA2ZtuDeUrikRDGDCI7clIMyhlUcDXeJV84zdUKpcvQLl3jjCPeynAFyYyem
nRgufzBOZMTuWpF21iaC7RPrYcA0pE83/kXrzGhWlIr2+A+15b3k+IvlsIFsLHCE
G3jKn9RbTHCRJ0pQnLds8KNpF1KnB0JW05FItx6a7QZFiotc/pmwz914uw2gB+Du
KOJjApcB21LwGxkQ+SN6r8v9pzyRfYV22PrEQlYtCMBQ0JnFTdd6m7GZUdUCAwEA
AaAAMA0GCSqGSIB3DQEBChUAA4IBAQCzKYTGzA/iAx8lv5yr0A8HwpEia2uc/4Q7
b33zm4XgkYxIJrWMTBZ7H8QWggsReeKmbk4FHxrG6n7r/PrgDCBL0lN1recdfJj
WeOA+EPm1xMCv9FJorsye67knhKT7SGEtWe6ZSzoTVLIJ3u5xiPL2rhcb0ineMDX
admIcJX0SNKIDTospyUzV0FzZTnhN8/Mb3Y+9THV5g+I3f5hvB5UgSp1H+6LIXb
INvNoy4HAMKFFkDMmAdP/ydmBErJvoMFY2PxVZ0MWhfnQw7FJfP0bckuSNSL+wB
9IKQpFr7UaNsAkn9aqy6ienGarjrwjJPFHVq8CFTPqf4sloxZyQh
-----END CERTIFICATE REQUEST-----
key_id   d09b8923-766f-6457-4222-9fbb02074f2d

```

```
debian@debian: ~  
2026-03-12T02:12:42.906-0500 [INFO] core: successful mount: namespace="" path=secret/ type=kv version="v0.25.0+builtin"  
WARNING! dev mode is enabled! In this mode, Vault runs entirely in-memory and starts unsealed with a single unseal key. The root token is already authenticated to the CLI, so you can immediately begin using Vault.  
  
You may need to set the following environment variables:  
  
$ export VAULT_ADDR='http://127.0.0.1:8200'  
  
The unseal key and root token are displayed below in case you want to seal/unseal the Vault or re-authenticate.  
  
Unseal Key: JmUrmQCwIDhMDCZevtPc4VKs5NT7d8HM9vjRB/h+NYk=  
Root Token: hvs.ziGwDo2h69kho82fSaSnLZML  
  
Development mode should NOT be used in production installations!  
  
2026-03-12T02:16:09.950-0500 [INFO] secrets.pki.pki_8a4861a9: 66566c0f-017a-4835-cf5b-490292fec299: succeeded in migrating to issuer storage version 2  
2026-03-12T02:16:09.950-0500 [INFO] core: successful mount: namespace="" path=pki/ type=pki version="v1.21.4+builtin.vault"  
2026-03-12T02:16:22.652-0500 [INFO] core: mount tuning of leases successful: path=pki/  
|  
  
debian@debian: ~  
root@debian:/home/debian# vault write pki/config/ca pem_bundle=@ca.crt  
WARNING! The following warnings were returned from Vault:  
  
* This mount hasn't configured any authority information access (AIA) fields; this may make it harder for systems to find missing certificates in the chain or to validate revocation status of certificates. Consider updating /config/urls or the newly generated issuer with this information.  
  
Key Value  
---  
existing_issuers <nil>  
existing_keys <nil>  
imported_issuers [1d323539-deaa-e681-1415-b90ee1d4efe9]  
imported_keys <nil>  
mapping map[1d323539-deaa-e681-1415-b90ee1d4efe9:]  
root@debian:/home/debian#
```

Configurando a intermediate.

```
GNU nano 7.2 /etc/apache2/sites-available/default-ssl.conf *  
#Include conf-available/serve-cgi-bin.conf  
  
# SSL Engine Switch:  
# Enable/Disable SSL for this virtual host.  
SSLEngine on  
  
# A self-signed (snakeoil) certificate can be created by installing  
# the ssl-cert package. See  
# /usr/share/doc/apache2/README.Debian.gz for more info.  
# If both key and certificate are stored in the same file, only the  
# SSLCertificateFile directive is needed.  
SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem  
SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key  
  
SSLCertificateFile /home/debian/app.crt  
SSLCertificateKeyFile /home/debian/app.key  
SSLCertificateChainFile /home/debian/ca_chain.crt
```

Substituindo os certificados do apache.

```

root@debian:/home/debian# openssl s_client -connect localhost:443
CONNECTED(00000003)
Can't use SSL_get_servername
depth=2 C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
verify return:1
depth=1 CN = Lab Intermediate CA
verify return:1
depth=0 CN = app.lab.local
verify return:1
---
Certificate chain
 0 s:CN = app.lab.local
  i:CN = Lab Intermediate CA
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Mar 12 09:15:39 2026 GMT; NotAfter: Mar 15 09:16:09 2026 GMT
 1 s:CN = Lab Intermediate CA
  i:C = br, ST = sp, L = sp, O = Internet Widgits Pty Ltd, CN = lab-root-ca
  a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Mar 12 09:09:26 2026 GMT; NotAfter: Mar 11 09:09:26 2031 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIDyzCCArOgAwIBAgIU0duej7qWPvE0EyyvImpbIK6Lpn9owDQYJKoZIhvcNAQEL
BQAwHjEcmBoGA1UEAxMTTGFiIEludGVybWVkaWFOZSBDQTAEFw0yNjAzMTIwOTE1
MzlaFw0yNjAzMTUwOTE2MDIaMBGxFjAUBGNVBAMTDwFwcC5sYWVubG9jYVwwggEi
MA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQDLHB0QnNVprw92YBp81Kw+pB4/
DaIPj3ttk9c1r46s9qAPbnbsDGZ8cYKH+7+nTq2yulX7ednyAtxBMjpPI0ZAeSTB
-----

```

Novo certificado aplicado no apache.

```

root@debian:/home/debian# curl http://127.0.0.1:8200/v1/pki_int/crl -o crl.pem
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 453 100 453 0 0 286k 0 --:--:-- --:--:-- --:--:-- 442k
root@debian:/home/debian# openssl crl -in crl.pem -text -noout
Certificate Revocation List (CRL):
  Version 2 (0x1)
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: CN = Lab Intermediate CA
  Last Update: Mar 12 09:53:14 2026 GMT
  Next Update: Mar 15 09:53:14 2026 GMT
  CRL extensions:
    X509v3 Authority Key Identifier:
      4B:04:8F:13:29:59:36:42:9A:49:45:49:C2:D4:EC:83:3E:94:07:94
    X509v3 CRL Number:
      3
  Revoked Certificates:
    Serial Number: 39DB9E8FBA963EF134132BC89A96C82BA9699FDA
    Revocation Date: Mar 12 09:53:14 2026 GMT
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      6d:68:bc:17:85:da:97:e2:bf:4c:93:54:af:51:fc:34:06:97:
      39:fa:4c:c7:98:ad:de:ef:6c:e9:e3:0f:8d:51:f4:38:95:31:
      88:2d:56:19:f5:1c:b5:30:0e:0b:11:57:1d:64:b9:70:fb:9b:
      25:4e:23:3b:0a:a4:43:a7:d3:f1:d5:b8:03:0c:31:c9:52:0b:
      1f:8c:63:e3:78:2a:5f:cb:81:fd:1c:5c:19:2a:ad:d7:1b:84:
      c9:2e:a7:11:7f:07:58:af:14:cc:05:c5:d9:4f:ef:fd:88:cb:
      22:d3:76:08:b3:02:1e:7e:49:f9:ee:f9:a8:63:14:cd:cd:6a:
      4c:a8:11:3b:25:f5:00:be:28:e9:93:9f:8b:ff:79:1e:a2:0f:
      cd:fb:43:2b:7d:a3:19:a8:ea:51:86:c5:54:32:57:0b:42:4e:
      69:39:29:a3:f0:d7:d5:83:3b:09:2d:3a:6d:2d:0c:80:9e:bb:
      fa:78:3a:10:bd:84:50:4e:d5:d6:a8:5a:41:08:f0:52:49:8d:
      18:af:8d:9d:66:26:69:b0:8c:1f:bb:71:53:d7:69:32:3e:e0:
      41:e6:98:27:90:88:30:f4:2e:13:5e:9d:1c:b7:6f:a2:ca:d6:
      76:1f:6f:ce:51:9e:50:5e:83:d9:f3:ff:3e:00:18:79:07:56:
      47:c5:ba:f6

```

```
debian@debian: ~  
root@debian:/home/debian# openssl x509 -in app.crt -noout -serial  
serial=39DB9E8FBA963EF134132BC89A96C82BA9699FDA  
root@debian:/home/debian# vault write pki_int/revoke \  
serial_number="39:db:9e:8f:ba:96:3e:f1:34:13:2b:c8:9a:96:c8:2b:a9:69:9f:da"  
Key Value  
----  
revocation_time 1773309194  
revocation_time_rfc3339 2026-03-12T09:53:14.510977457Z  
state revoked  
root@debian:/home/debian#
```

Revogando certificado.

```
debian@debian: ~  
2026-03-12T02:12:42.903-0500 [INFO] identity: entities restored  
2026-03-12T02:12:42.903-0500 [INFO] identity: groups restored  
2026-03-12T02:12:42.903-0500 [INFO] expiration: lease restore complete  
2026-03-12T02:12:42.903-0500 [INFO] core: post-unseal setup complete  
2026-03-12T02:12:42.903-0500 [INFO] core: vault is unsealed  
2026-03-12T02:12:42.906-0500 [INFO] core: successful mount: namespace="" path=secret/ type=kv version  
="v0.25.0+builtin"  
WARNING! dev mode is enabled! In this mode, Vault runs entirely in-memory  
and starts unsealed with a single unseal key. The root token is already  
authenticated to the CLI, so you can immediately begin using Vault.  
  
You may need to set the following environment variables:  
  
$ export VAULT_ADDR='http://127.0.0.1:8200'  
  
The unseal key and root token are displayed below in case you want to  
seal/unseal the Vault or re-authenticate.  
  
Unseal Key: JmUrmQCwIDhMDCZevtPc4VK55NT7d8HM9vjRB/h+NYk=  
Root Token: hvs.ziGwDo2h69kho82fSaSnLZML  
  
Development mode should NOT be used in production installations!  
  
debian@debian: ~  
root@debian:/home/debian# export VAULT_ADDR='http://127.0.0.1:8200'  
root@debian:/home/debian# export VAULT_TOKEN="hvs.ziGwDo2h69kho82fSaSnLZML"
```

```

root@debian:/home/debian# vault write pki_int/issue/web \
common_name="app.lab.local"
WARNING! The following warnings were returned from Vault:

* TTL "768h0m0s" is longer than permitted maxTTL "72h0m0s", so maxTTL is
being used

* TTL "768h0m0s" is longer than permitted maxTTL "72h0m0s", so maxTTL is
being used

Key                                     Value
---                                     -
authority_key_id                       4b:04:8f:13:29:59:36:42:9a:49:45:49:c2:d4:ec:83:3e:94:07:94
ca_chain                               [-----BEGIN CERTIFICATE-----
MIIDYDCCAkigAwIBAgICEAEwDQYJKoZIhvcNAQELBQAwYDELMAkGA1UEBhMCYnIx
CzAJBgNVBAGMAmNwMQswCQYDVQQHDAJzcDEhMB8GA1UECgwYSW50ZXJuZXQgV2lk
Z2l0cyBQdHkgTHRkMRQwEgYDVQQDDAsYWItdcm9vdC1jYTAeFw0yNjAzMTIwOTA5
MjZaFw0zMTAzMTEwOTA5MjZaMB4xHDAaBgNVBAMTE0xhYiBJbnRlcm1lZGhhdGUg
Q0EwggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC0eaCYWj1z83f8s/WS
fSOvgrj/Z53/FwohLERT7RLP2WQKYqWE5LMb6bd9+kuT6THPavrvDV1hxjBWq3uAp
MhgwYHsvBzM2W8cANE9KH89xfQM0nI4e7c5JrBTfSoniLQLCuFAKxh2t7B0XISsd
t5rEz3YzDz0T91bsOK4Q++6dSwIujXZY0Xy/C3wOdMXIrk7IYJJx0DZs92SmQKrZ
ZScSBe6W9vd0vTmVLAYbONay09IUp1FsFWE5sQs2qFbK3SS2iXJ2pKGkPj+lspC5
wXjIKes82ggM0z5iTfYRZWg2v63INvOGiW0bSB7pt7KhNtaY5pS22EzL4qie0puN
cFZJAgMBAAGjZjBkMBIGA1UdEwEB/wQIMAYBAf8CAQAwDgYDVDR0PAQH/BAQDAgEG
MB0GA1UdDgQWBRLBI8TKV/k2QppJRUnC10yDPpQHLDaFBgNVHSMEGDAWgBSmaE6d
pjmD6J8jbitAQ58x4whbaDANBgkqhkiG9w0BAQsFAAOCAQEAgg/NP19oLHlXik2Q
UEUD4urqQmTK35dHw25IcFyeNy6hqnVIUULT60Z5MzsRNdCT0C0Y20QE0QVstpa
fGw0F7iWUFH+od1nefM3ymMnIQX8Ln05mYdDeWcon1ohu5M9vmmF6XfTRPNxYtbI
NQywyXGkH3TI9wDvgBkvsXjhK7t44Q/GYve+VLLbjxySkkR6I5/sJQaWb0AKKPR
iLNqqvtplpn6VseSg6GAqexGX5ukaYkoT0Z2/Qjmcx0rNRXUGco5FqemNb45sBU
cVdEPheTONx0AMEKHUYVmLYQRoutVKm43Zh00ohnOMhpoYKCIatowQ7WQKaEfM4w
12GZUQ==
-----END CERTIFICATE-----]

```

Gerando certificado.

7. Conclusão:

Ao final deste laboratório, foi possível compreender e implementar os principais componentes de uma infraestrutura de certificação digital. A utilização do SoftHSM permitiu simular o funcionamento de um HSM, garantindo que a chave privada da Autoridade Certificadora fosse armazenada e utilizada de forma segura por meio do padrão PKCS#11. Com o auxílio do OpenSSL, foi possível gerar solicitações de certificado, criar uma CA e assinar certificados para uso em um servidor web seguro configurado no Apache HTTP Server.

Além disso, a introdução do HashiCorp Vault demonstrou como ferramentas modernas podem ser utilizadas para centralizar e automatizar o gerenciamento de segredos e certificados, aumentando a segurança e reduzindo a complexidade operacional em ambientes de produção. Dessa forma, o laboratório permitiu consolidar conhecimentos fundamentais sobre PKI, proteção de chaves criptográficas e gestão de certificados digitais, conceitos amplamente utilizados em infraestruturas corporativas, serviços em nuvem e arquiteturas de segurança modernas.